REFLECTION QUESTIONS

1. Experiential Reflection: What was the most challenging defect to locate inside the legacy codebase, and how did systematic unit testing assist you in pinpointing the root cause compared to random debugging?

- The most challenging defect was finding an error caused by moving elements in the array. Systematic unit testing helped because I could test one small part of the code at a time. This made it easier to see exactly where the problem happened. Random debugging they do in an array. It also does not need to create a bigger buffer each time I will use clear variable and function names, add useful comments, and keep my code organized. I will also test my code often and it would take more time because I would be guessing where the error was.

2. Architectural Evaluation: Knowing what you now know about dynamic array reallocations and shift-based element deletions, what linear architecture alternative (e.g., Linked Lists, Queues, Stacks) would you recommend for systems that require frequent mid-list insertions without reallocating buffers?

- I would recommend using a linked list. A linked list is better for frequent insertions in the middle because elements do not need to be moved or copied like they do in an array. It also does not need to create a bigger buffer each time the array becomes full. This can make the program more efficient when many insertions are needed.

3. Professional Growth: How has evaluating another programmer's code during the Peer Code Review altered your approach to writing clean, well-documented C++ code for your upcoming Milestone 1 project?

- Reviewing another programmer's code showed me the importance of writing code that is easy to understand. For my Milestone 1 project, I will use clear variable and function names, add useful comments, and keep my code organized. I will also test my code often instead of waiting until the end to find mistakes.